

PATRONES DE DISEÑO

EVALUACIÓN DE LA UNIDAD III

Nombre del Alumno		Fecha de observación (dd/mm/aaaa)	
Carrera/ Sede			
Sección/ Jornada			
Evidencia a evaluar	Implementación de algoritmos según requerimientos dados. Todos los aprendidos en la unidad		
Aprendizaje(s) esperado(s)	Todos los abordados en la unidad		

Puntaje total	Puntaje de corte	Puntaje obtenido	Nota	Ponderación
45 puntos	30 puntos			33% (práctico)

Niveles de desempeño:

Cumple completamente	Cumple con todos los aspectos solicitados en el indicador.
Cumple parcialmente	Cumple de manera parcial los requisitos del indicador.
Insuficiente	La evidencia es inconsistente y no es suficiente para evidenciar el logro del indicador.
Ausente	La evidencia no cumple los requisitos del indicador o no se presenta.

Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota
0.0	1.0	10.0	2.0	20.0	2.9	30.0	3.9	40.0	5.9
1.0	1.1	11.0	2.0	21.0	3.0	31.0	4.0	41.0	6.1
2.0	1.2	12.0	2.1	22.0	3.1	32.0	4.1	42.0	6.3
3.0	1.3	13.0	2.2	23.0	3.2	33.0	4.3	43.0	6.6
4.0	1.4	14.0	2.3	24.0	3.3	34.0	4.6	44.0	6.8
5.0	1.5	15.0	2.4	25.0	3.4	35.0	4.8	45.0	7.0
6.0	1.6	16.0	2.5	26.0	3.5	36.0	5.0		
7.0	1.7	17.0	2.6	27.0	3.6	37.0	5.2		
8.0	1.8	18.0	2.7	28.0	3.7	38.0	5.4		
9.0	1.9	19.0	2.8	29.0	3.8	39.0	5.7		

Criterios de Evaluación

Criterios	Niveles de desempeño			
	Cumple completamente	Cumple parcialmente	Insuficiente	Ausente
1. Patrones Creacionales (Singleton y Factory Method)	Singleton es inaccesible mediante new y maneja concurrencia (lock). Factory Method oculta por completo la instanciación de SqlConnection y la creación de Jugadores; el controlador y repositorio operan sólo con creadores abstractos.	Los objetos se crean, pero hay fugas de diseño: el Singleton no es Thread-Safe, o el cliente (Controlador/Repositorio) todavía hace referencias directas a clases concretas o usa new SqlConnection().	No se aplican las estructuras de los patrones; la creación se resuelve de forma rígida y directa en el código del controlador o repositorio.	No se evidencia ninguna estructura orientada a la creación abstracta de objetos o el código no compila.
2. Patrón de Comportamiento (Iterator)	Implementa correctamente IJugadorIterator. El iterator encapsula la colección obtenida de la BD sin exponer su estructura. El controlador extrae los datos usando exclusivamente TieneSiguiente() y Siguiente().	El iterator está creado e implementa la interfaz, pero expone la lista interna al cliente, o el controlador extrae los datos de forma mixta (saltándose el iterator en algunas partes).	Se creó una clase iteradora, pero no resuelve el problema; el controlador sigue manipulando la colección de la base de datos de forma directa y tradicional.	No se implementó el patrón o no se integra en absoluto en el recorrido de la plantilla.
3. Patrón Estructural (Adapter)	El adaptador envuelve perfectamente la clase simulada (Legacy). Implementa la interfaz moderna IClimaTorneo, extrae el texto, ejecuta la conversión matemática a Celsius y entrega un entero limpio a la vista.	Se logra la conectividad a la vista, pero violando el patrón: se modificó el código interno de la clase Legacy, o la lógica de extracción y conversión (Parse) quedó expuesta en el controlador.	Se creó una clase "Adapter", pero no resuelve la incompatibilidad; el controlador sigue acoplado a la cadena de texto antigua de grados Fahrenheit.	No se implementó el patrón o no se integra en absoluto en el módulo de logística.
4. Integración de Capas (Backend-Frontend)	El cliente interactúa con el sistema utilizando únicamente las interfaces y abstracciones definidas por los patrones, demostrando el bajo acoplamiento del diseño general.	El flujo funciona, pero el código cliente rompe el encapsulamiento de algún patrón en puntos específicos, manipulando directamente objetos concretos que debieron estar ocultos.	El cliente está fuertemente acoplado a todas las implementaciones; la simulación falla o los patrones se ejecutan como islas sin conexión ni coherencia arquitectónica.	El programa principal no demuestra la interacción de las estructuras o no compila.
5. Aseguramiento de Calidad (Pruebas Unitarias)	Implementa correctamente las dos pruebas exigidas (MSTest o xUnit): Valida de forma efectiva que el Singleton mantenga la misma referencia en memoria y que el Adapter convierta el texto crudo a Celsius con precisión.	Existe el proyecto de pruebas, pero solo implementó correctamente un escenario, o las pruebas están redactadas pero contienen fallos lógicos que impiden su validación real.	El proyecto de pruebas existe pero evalúa lógicas irrelevantes del sistema, o las aserciones (Assert) no tienen relación con los patrones de diseño evaluados.	No se entregó el proyecto de pruebas unitarias o los tests no fueron implementados.

Enunciado del Problema

Misión: Han sido designados como Arquitectos de Software por el comité organizador del Mundial 2026. El equipo de Frontend ha entregado la interfaz gráfica terminada y el equipo de Datos ha montado las tablas en SQL Server.

Actualmente, el Motor Central (Backend) carece de una arquitectura limpia. Su objetivo es **construir la capa de negocio** aplicando cuatro Patrones de Diseño clave (Factory Method, Singleton, Iterator y Adapter) para estructurar el sistema y conectarlo con la base de datos de manera escalable, mantenible y profesional.

Reglas de Oro (Contratos de Desarrollo)

Para aprobar esta evaluación y construir una base sólida en su formación como desarrolladores, deben adherirse estrictamente a las siguientes directrices arquitectónicas:

1. **El Contrato Visual (Zona Restringida):** Está **ESTRICTAMENTE PROHIBIDO** modificar los archivos HTML/CSS/Razor de la carpeta Views. Su código backend debe adaptarse a la vista provista. Alterar la UI invalidará la evaluación del módulo.

Estructura Arquitectónica del Proyecto

Deberán trabajar exclusivamente dentro de la carpeta Patrones y realizar las inyecciones correspondientes en el Controllers/HomeController.cs y Repository/ChannelRepository.cs.

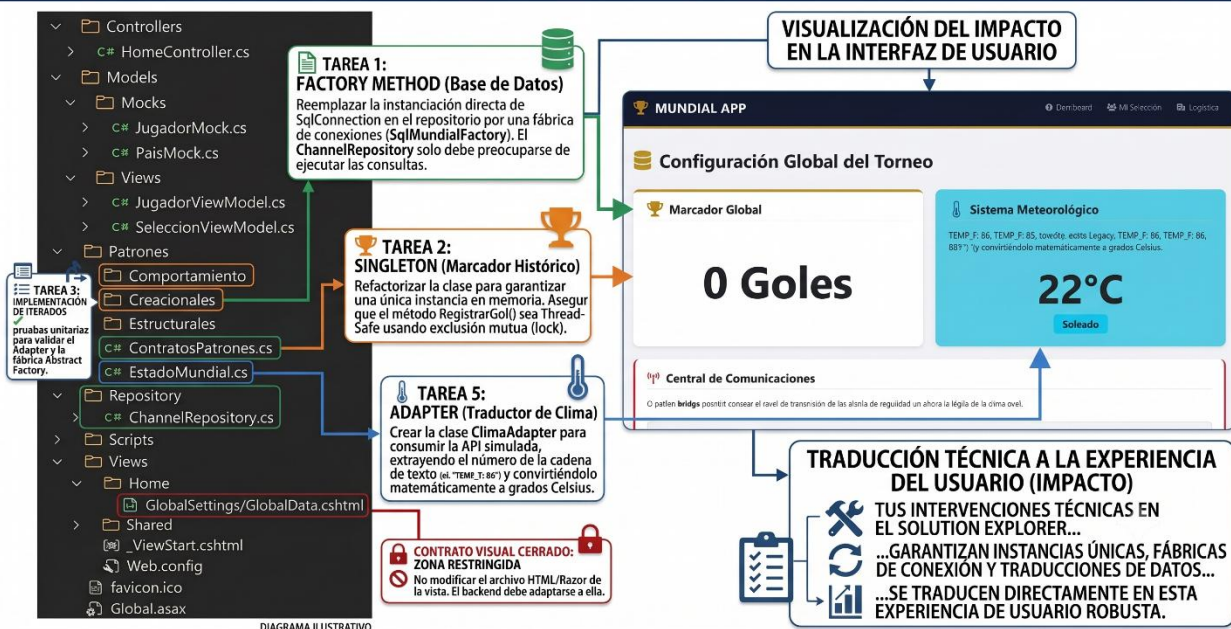
```
Santotomas.Web.MundialEVFinal
|
|— Controllers
|   |— HomeController.cs    <-- [INTERVENCIÓN] Conecten sus patrones aquí.
|— Patrones                <-- ## ÁREA DE TRABAJO PRINCIPAL ##
|   |— Comportamiento
|   |— Creacionales
|   |— Estructurales
|   |— ContratosPatrones.cs <-- Interfaces base.
|   |— EstadoMundial.cs
|— Repository
|   |— ChannelRepository.cs <-- [INTERVENCIÓN] Refactorizar la conexión aquí.
|— Views                    <-- ZONA RESTRINGIDA – No Modificar
|   |— Home (Index, MiEquipo, Logistica)
```

Zona de Intervención de la Solución

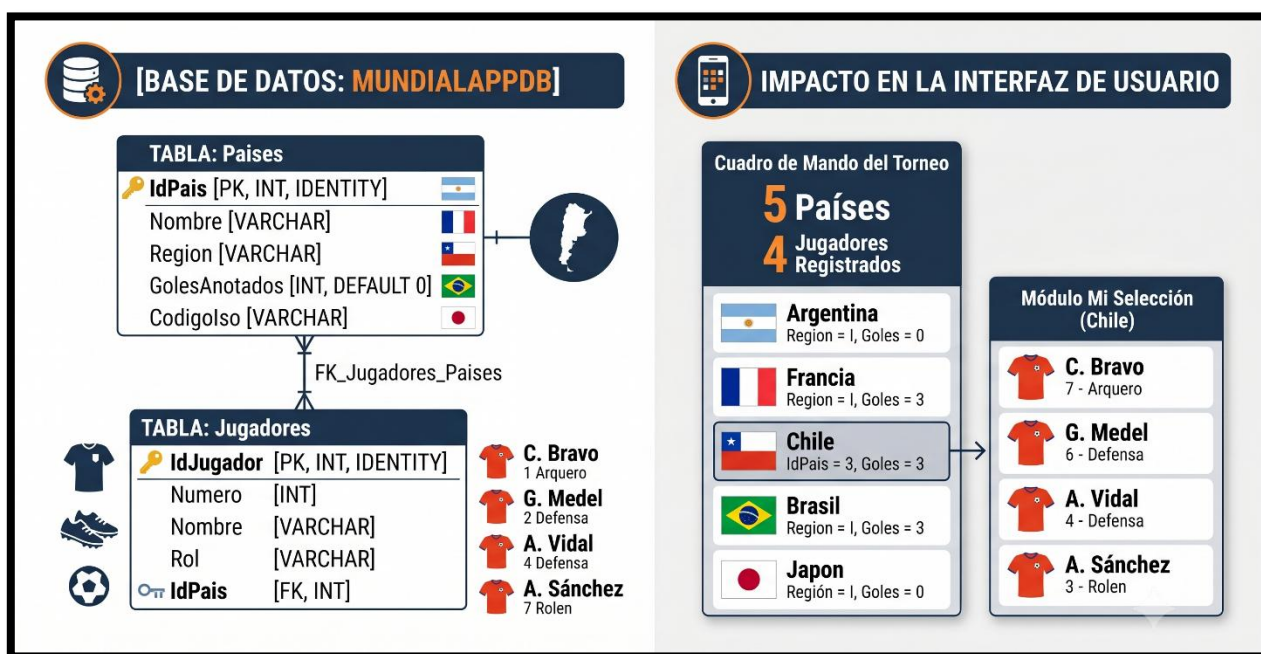
GUÍA DE INTERVENCIÓN: MÓDULO MI SELECCIÓN (IMPLEMENTACIÓN DE PATRONES)



GUÍA DE INTERVENCIÓN: CONFIGURACIÓN GLOBAL Y DATOS



Modelo Datos



Indicaciones

PASO 1: Motor de Conexiones a Base de Datos (Patrón Factory Method)

- **Objetivo:** Desacoplar el repositorio de la instancia directa de la base de datos y centralizar la lectura de la cadena de conexión.
- **Ubicación:** Patrones/Creacionales y Repository/ChannelRepository.cs
- **Instrucciones:**
 1. En ContratosPatrones.cs, crear la interfaz IConexionFactory con un único método llamado IDbConnection CrearConexion().
 2. Crear la clase SqlMundialFactory que implemente esta interfaz. Su responsabilidad será leer la cadena de conexión desde el archivo Web.config, instanciar el objeto SqlConnection, y retornarlo.
 3. En el archivo ChannelRepository.cs está **prohibido** utilizar new SqlConnection(). El repositorio debe utilizar la fábrica y su método .CrearConexion() cada vez que necesite comunicarse con SQL Server.

PASO 2: El Marcador Global (Patrón Singleton)

- **Objetivo:** Instancia única y segura en memoria.
- **Ubicación:** Patrones/EstadoMundial.cs
- **Instrucciones:**
 1. Asegurar que el constructor de la clase EstadoMundial sea privado.
 2. Programar la propiedad de instancia para que, en su primera ejecución, invoque a ChannelRepository y cargue la sumatoria histórica de goles directamente desde la base de datos.
 3. Utilizar exclusión mutua (lock) en el método RegistrarGol() para garantizar la seguridad de hilos (Thread-Safety).

PASO 3: Reclutamiento Táctico (Patrón Factory Method)

- **Objetivo:** Instanciar jugadores polimórficamente sin usar condicionales.
- **Ubicación:** Patrones/Creacionales y HomeController.cs
- **Instrucciones:**
 1. Crear las clases ArqueroFactory y DelanteroFactory implementando la interfaz IJugadorFactory. Cada fábrica debe instanciar a su respectivo jugador.
 2. En el controlador, reemplacen los retornos nulos de los botones de convocatoria inyectando estas fábricas en un Diccionario para despachar la creación dinámica.

PASO 4: Recorrido de Plantilla (Patrón Iterator) – Obtener Información de BD

- **Objetivo:** Leer la base de datos de jugadores sin exponer la estructura interna de la colección.
- **Ubicación:** Patrones/Comportamiento y HomeController.cs
- **Instrucciones:**
 1. Crear la clase PlantillaIterator basada en la interfaz IJugadorIterator. Debe encapsular la colección de jugadores obtenida desde el repositorio.
 2. Refactorizar la acción de la grilla en el controlador para que la vista se pueble utilizando únicamente los métodos TieneSiguiente() y Siguiente().

PASO 5: Sistema Meteorológico (Patrón Adapter)

- **Objetivo:** Traducir datos de un sistema externo incompatible (Legacy) para que nuestra vista moderna pueda consumirlos.
- **Ubicación:** Patrones/Estructurales
- **Instrucciones:**
 1. **Simulación del Sistema Incompatible (100% FICTICIO):** Creen una clase sencilla llamada ServicioClimaAntiguo.
NOTA ACLARATORIA: No deben consumir ninguna API web real ni conectarse a internet. Esta clase es solo un simulador local (Mock). Únicamente debe contener un método básico que retorne la cadena de texto estática "TEMP_F: 86".
 2. Asegúrense de tener la interfaz IClimaTorneo con el método int ObtenerTemperaturaCelsius().
 3. Creen la clase ClimaAdapter que implemente IClimaTorneo. El adaptador debe instanciar a ServicioClimaAntiguo, extraer el número "86" de la cadena de texto, aplicar la fórmula matemática para convertirlo a Celsius y entregar este valor numérico limpio a la vista del estadio.

PASO 6: Aseguramiento de Calidad (Pruebas Unitarias)

Ubicación: Proyecto Santotomas.Web.MundialEVFinal.Tests

Deberán programar un mínimo de **dos pruebas unitarias** utilizando MSTest o xUnit:

1. **Validación de Instancia Singleton:** Verifiquen mediante un Assert.AreSame que dos peticiones distintas a la clase EstadoMundial retornan exactamente el mismo espacio en memoria.

2. **Validación de Adaptación:** Verifiquen que si su adaptador recibe de la clase simulada la cadena "TEMP_F: 50", el método procesa correctamente el texto y retorna el número entero 10.

Criterios de Entrega

- **Formato:** Solución completa comprimida en .zip o enlace a repositorio de control de versiones.
- **Compilación:** El proyecto debe compilar a la primera. Proyectos que no compilen serán calificados con la nota mínima.
- **El proyecto puede ser entregado a través de las plataformas:** Aula Santo Tomas, Sharepoint Institucional o GIT.

Plazo de Entrega: Martes 30 de Junio - 2026.

Importante: Queda Prohibida la utilización de IA's.

Formato: APELLIDOS_NOMBRES.ZIP

Dentro del ZIP, otorgar un Archivo TXT con los integrantes